PRODUCT REQUIREMENTS DOCUMENT · PHASED BUILD GUIDE

# Detecting AI-Generated Images Using Frequency-Domain Features

A step-by-step plan that Codex can execute one phase at a time — and that the team can read to understand exactly how the project gets built. Every tool used is on a free tier.

| Item | Detail |
|---|---|
| Team | Rahul Temkar · Shravani Rane · Jagruti Doke (MET Institute of Computer Science, MCA) |
| Based on | Proposal Draft & Scope Definition v0.1 and Assignment 4 — Methodology & Analysis |
| Version / date | PRD v1.0 · 26 October 2026 |
| Build agent | OpenAI Codex (ChatGPT Free plan), running one phase per session with a STOP gate after each |
| Compute | Google Colab free tier (CPU is enough) + Google Drive; laptop for the local prototype |
| Cost | ₹0 — no paid service, API key, GPU or hosting anywhere in the plan |

**How to read this document.** Sections 1–5 explain *what* is built and *with what*. Section 6 is the build itself: ten phases (0–9). Each phase has (1) a plain-English explanation for the team, (2) a copy-paste prompt for Codex, (3) checks you run yourself, and (4) a STOP gate. Section 7 lists the difficulties you will face, ranked by how badly they would hurt the result, each with a fix. Labels used: **Likely:** strong evidence but not verified; **Assumption:** chosen to proceed, could be wrong. Unlabelled statements were verified or follow by definition.

## Contents


| | |
|---|---|
| 1 | One-page summary |
| 2 | How the Codex workflow works (and the STOP protocol) |
| 3 | Product requirements |
| 4 | Free-tier stack |
| 5 | Dataset decision |
| 6 | Build phases 0–9 with Codex prompts |
| 7 | Difficulties you will face — and fixes |
| 8 | Statistics quick reference |
| 9 | Decisions to confirm with the supervisor |
| 10 | Sources checked |

# 1. One-page summary

**What gets built:** a Python package that turns any image into a fixed-length frequency-feature vector (2D FFT → high/low energy ratio, spectral entropy, 64-ring radially averaged power spectrum, peak count/strength), a matched pixel-only baseline, three tuned classifiers (SVM, Random Forest, XGBoost) for each feature set, a one-time held-out test with the four pre-registered statistical tests, and a local Streamlit app that returns “Real” or “AI-generated” with a calibrated score.

**How it gets built:** Codex writes the code and unit tests in a GitHub repository, one phase per session. Codex tests everything on small synthetic images it generates itself. The team then runs the real-data steps in a free Colab notebook that clones the repo. After every phase Codex stops and writes a report; nobody moves on until the phase checks pass.

| Phase | Name | Proposal weeks | Main output |
|---|---|---|---|
| 0 | Repository, rules and environment | 1 | Repo skeleton, AGENTS.md, config.yaml, CI tests |
| 1 | Data download (pilot + main) | 1 | CIFAKE pilot sample; SD v1.5 subset of Tiny-GenImage on Drive |
| 2 | Data audit, de-duplication, frozen splits | 2 | audit report, shortcut probe, splits.csv + hash |
| 3 | Shared preprocessing function | 3 | preprocess.py + tests (one function for everything) |
| 4 | Frequency feature module (D1) | 4–5 | features.py + tests, example spectra |
| 5 | Pixel baseline + feature extraction | 6 | train/val/test feature files (test sealed) |
| 6 | Training and tuning | 7–8 | 6 tuned models, CV results, chosen $r_0$ and peak threshold |
| 7 | Freeze, single test run, hypothesis tests (D2) | 9 | metrics tables, binomial/McNemar/Cochran Q results |
| 8 | Streamlit prototype (D3) | 10–11 | app.py running locally |
| 9 | Report pack and reproducibility (D4 support) | 12 | figures, tables, model card, README |

**Two changes to the earlier drafts that matter most.** (1) **Main dataset:** use the Stable Diffusion v1.5 subset of Tiny-GenImage instead of CIFAKE, because CIFAKE images are 32 × 32 and cannot carry the high-frequency traces the project is looking for; keep CIFAKE only as a 1-day pilot. (2) **Shortcut probe:** a published study showed that detectors trained on GenImage learn JPEG compression and image size instead of generation traces, so Phase 2 adds a test that must pass before any frequency feature is computed. Section 5 and Difficulty #1 explain both.

# 2. How the Codex workflow works

Codex is an AI coding agent. It reads your repository, writes code, runs commands and tests, and proposes changes. It works best with small, checkable tasks and a written rulebook. This plan gives it both: the rulebook is **AGENTS.md** (Codex reads this file automatically from the repository root), and the tasks are the ten phase prompts in Section 6.

## 2.1 Where each kind of work happens

| Work | Who / where | Why |
|---|---|---|
| Writing code, unit tests, docs | Codex, inside the GitHub repo | Codex can run pytest on synthetic data it creates itself |
| Downloading data, auditing, extracting features, training, testing | You, in a free Colab notebook that clones the repo and calls its scripts | The data (~1–2 GB) lives on your Google Drive, not in Codex's sandbox |
| Checking results, deciding to proceed | The three team members | Codex never decides that a phase passed |
| Running the prototype | Your laptop (Python + Streamlit) | Scope says local demo only; free |

## 2.2 The loop for every phase

- **Start a fresh Codex task** and paste the phase prompt from Section 6 (each prompt is self-contained).
- **Codex works** on a branch named `phase-N`, writes code + tests, runs `pytest`, and writes `reports/phase_N_report.md`.
- **Codex stops.** It must not start the next phase on its own (rule in AGENTS.md).
- **You review**: read the report, open the pull request, run the Colab cell listed under "Checks".
- **If a check fails**, paste the exact error or the failing number back to Codex in the same task: "Check 3 failed: <output>. Fix only this."
- **When all checks pass**, merge the PR, tick the phase in the team tracker, and open a new task with the next prompt, starting with "PROCEED TO PHASE N+1".

## 2.3 Phase report template (Codex fills this in every time)

```
# Phase N report
## Done
- files created/changed (one line each, with purpose)
## Tests
- command run, number passed/failed, paste the summary line
## Outputs produced (paths)
## Checks for the team to run in Colab (exact cells/commands)
## Assumptions I made (each: what I assumed, what changes if wrong)
## Open questions / anything I could not verify
## STOP — waiting for "PROCEED TO PHASE N+1"
```

## 2.4 AGENTS.md — paste this into the repository root in Phase 0

```
# AGENTS.md — rules for every Codex task in this repository
Project: detect AI-generated vs real images with hand-crafted 2D-FFT features + SVM / RF / XGBoost.

## Working rules
1. Do exactly ONE phase per task. When the phase is finished, write reports/phase_N_report.md using
   the template in docs/PHASE_REPORT_TEMPLATE.md, commit on branch phase-N, and STOP.
   Never start the next phase until the user writes "PROCEED TO PHASE N+1".
2. Free tier only: no paid APIs, no API keys, no cloud services other than GitHub, Google Colab,
   Google Drive, Kaggle and Hugging Face public downloads. No GPU code is needed.
3. All settings live in config.yaml (seed=42, image_size, K rings, paths). No magic numbers in code.
4. ONE preprocessing function (src/aidetect/preprocess.py: preprocess_image) is used for training,
   evaluation and the prototype. Never write a second, class-specific or app-specific version.
5. THE TEST SET IS SEALED. No code may read data/splits test rows or features/test.* unless
   the file artifacts/FREEZE.lock exists AND the command is scripts/evaluate_test.py.
6. Every function gets a pytest test. Tests must not need the real dataset: generate tiny
   synthetic images (noise, sinusoids, checkerboards) inside the tests.
7. Do not guess library parameters. If unsure of a function signature, check with help() or the
   installed package and say in the report what you checked.
8. If a step fails or a test cannot pass, stop and report which step failed and why.
   Do not silently work around it.
9. Pin versions in requirements.txt. Python 3.10+. Use numpy, scipy, pillow, scikit-learn, xgboost,
   statsmodels, pandas, pyarrow, imagehash, matplotlib, datasets, streamlit, joblib, pytest.
10. Positive class = AI-generated (label 1). Real = 0. Use this everywhere.
```

# 3. Product requirements

## 3.1 Goal and non-goals

**Goal:** answer RQ1–RQ3 with a valid test — whichever way the answers go — and demonstrate the best model in a local app. **Non-goals** (from Section 13 of the proposal): cross-generator testing, robustness to social-media compression, deep-learning detectors, colour/wavelet/DCT/noise-residual features, localisation of edited regions, metadata analysis, video, adversarial attacks, production hosting.

## 3.2 Users

| User | Needs |
|---|---|
| Team (builders) | Clear phases, reproducible runs, no surprise costs |
| Supervisor / examiner | Evidence the result is real: audit, frozen splits, single test run, statistics with decisions |
| Demo viewer | Upload one image, see a label and a score, understand it is a model score, not proof |

## 3.3 Functional requirements

| ID | Requirement | Verified by |
|---|---|---|
| FR-1 | Load the chosen dataset and write a manifest (id, path, label, generator, format, width, height, file size, JPEG quality estimate) | Phase 1–2 checks |
| FR-2 | Detect exact and near-duplicates (perceptual hash) and keep each duplicate group in one split | Phase 2 test |
| FR-3 | Produce stratified 70/15/15 splits once, save to splits.csv, store its SHA-256 hash | Phase 2 |
| FR-4 | Shortcut probe: a classifier using only file metadata must not beat chance meaningfully (rule in Phase 2) before the project continues | Phase 2 |
| FR-5 | preprocess_image(): decode → greyscale → centre-crop → resize (one method) → [0,1] float | Phase 3 tests |
| FR-6 | extract_frequency_features(): fixed-length vector of 1 + 1 + K + 2 values (K = 64 → 68 values) | Phase 4 tests |
| FR-7 | extract_pixel_features(): 32 × 32 greyscale → 1,024 values | Phase 5 tests |
| FR-8 | Train SVM (RBF), RF, XGBoost on both feature sets with the same CV folds and equal search budget | Phase 6 |
| FR-9 | Write FREEZE.lock (hashes of config, code, models) before the test set is read; evaluate once | Phase 7 |
| FR-10 | Compute accuracy + 95% CI, precision, recall, F1, ROC-AUC, confusion matrix for all 6 combinations | Phase 7 |
| FR-11 | Run the binomial, McNemar (+Holm) and Cochran's Q (+pairwise McNemar, Holm) tests with decisions at α = 0.05 | Phase 7 |
| FR-12 | Streamlit app: upload one image → shared preprocessing → best model → label + calibrated score; reject corrupt/unsupported files | Phase 8 |

## 3.4 Non-functional requirements

- **Cost:** ₹0. Any step that would need payment is a defect.
- **Reproducibility:** fixed seed 42; versions pinned; every output file records the config hash it came from.
- **Determinism:** running feature extraction twice on the same image gives identical numbers (tested).
- **Runtime:** Assumption: full feature extraction for ~4,000–5,000 images finishes within one Colab session on CPU. If wrong, process in batches of 500 with resume (built in from Phase 5 anyway).

- **Honesty:** null and mixed results are reported in full; changes after opening the test set are labelled exploratory.

## 4. Free-tier stack

| Tool | Used for | Free-tier status (checked 26 Oct 2026) |
|---|---|---|
| OpenAI Codex via ChatGPT Free | Writing code and tests, phase by phase | OpenAI's help centre states Codex is included in ChatGPT plans including Free, with usage limits that vary by plan. Free limits are tight — see Difficulty #8. |
| GitHub (free account) | Repository, branches, pull requests, Actions for pytest | Likely: free private repos and Actions minutes are enough for pytest runs — based on GitHub's long-standing free plan. If Actions minutes run out, run pytest in Colab instead. |
| Google Colab (free) | Running data, feature, training and test scripts | Free, but Google's FAQ says resources are not guaranteed and limits fluctuate. Likely: ~12 h max session, ~12–13 GB RAM, idle disconnects — based on several independent 2026 guides. This project needs CPU only. |
| Google Drive (free) | Storing data subset, features, models | Likely: 15 GB free per Google account. The plan needs ~2–3 GB. |
| Hugging Face Hub | Downloading Tiny-GenImage (public parquet) | Public download, no payment. Dataset licence CC BY-NC-SA 4.0 (academic, non-commercial: OK). |
| Kaggle (free) | Downloading CIFAKE for the pilot; backup notebook if Colab is busy | Free account; notebooks free with session limits. |
| Python libraries | numpy, scipy, scikit-learn, xgboost, statsmodels, pillow, imagehash, streamlit… | Open source. statsmodels functions mcnemar(table, exact, correction) and cochrans_q(x) checked. |
| Streamlit (local) | Prototype on a laptop | Open source; runs locally with `streamlit run app.py`. No hosting needed. |

# 5. Dataset decision

| | CIFAKE (pilot only) | Tiny-GenImage, SD v1.5 subset (main) |
|---|---|---|
| Source | Kaggle | Hugging Face: TheKernel01/Tiny-GenImage (scaled-down GenImage) |
| Real images | CIFAR-10 photos | ImageNet photos |
| AI generator | Stable Diffusion 1.4 | Stable Diffusion v1.5 (filter generator == SD15, label value 6) |
| Resolution | 32 × 32 (from the proposal) — too small for frequency work | Real images: median side 500 px in the dataset statistics. Likely: SD images ~512 × 512 — based on the "Fake or JPEG?" paper noting SD images are similar in size to the real ones. |
| Size | 60k + 60k | Whole dataset 35,000 images (28,000 train + 7,000 validation), 8.36 GB download. Likely: ~1,500–2,200 SD v1.5 images — based on Hugging Face partial statistics showing 1,571 per generator in the sampled train rows. Phase 1 counts the exact number. |
| Role | 1-day pilot to test the pipeline end-to-end | All reported results |

**Why SD v1.5 and not another generator in the same dataset.** Image size is a known shortcut: BigGAN images are 128 × 128 and ADM 256 × 256, so a detector can learn "small image = fake". Because SD images are close in size to the ImageNet photos, both classes are shrunk by a similar factor when resized to 256 × 256, which removes most of the size shortcut. SD v1.4 would be equally good, but the partial statistics showed 0 SD14 rows in the sampled train data, so Phase 1 checks it and SD v1.5 is the default.

**Planned sample.** Assumption: about 2,000 AI images + 2,000 randomly drawn real images. A 15% test split is then ~600 images, and $H_0$ is rejected if at least 321 are correct (53.5%) — computed with the exact binomial and checked against the normal approximation. If the subset is smaller, the bar rises (Section 8). If the team needs more images, the documented fallback is the full GenImage SD v1.4 subset (very large) — a scope change that needs supervisor approval (proposal Section 18).

> **Limitation to state in the report:** the Hugging Face copy has no ImageNet class field, so per-category accuracy (proposal risk "content mismatch") cannot be reported. GenImage generated its AI images from the same 1,000 ImageNet class labels, which keeps content roughly matched, but this is not verified per image.

# 6. Build phases 0–9

Each phase: **In plain words** (for the team) → **Codex prompt** (copy-paste) → **Checks** (you run them) → **STOP gate**. Repository layout created in Phase 0:

```
ai-image-fft-detector/
  AGENTS.md            config.yaml          requirements.txt        README.md
  docs/PHASE_REPORT_TEMPLATE.md
  src/aidetect/  __init__.py  io_utils.py  data.py  audit.py  splits.py  preprocess.py
                 features.py  pixel_baseline.py  train.py  evaluate.py  stats_tests.py  plots.py
  scripts/  download_pilot.py  download_main.py  run_audit.py  make_splits.py  extract_features.py
            train_models.py  freeze.py  evaluate_test.py  make_figures.py
  tests/   test_preprocess.py  test_features.py  test_splits.py  test_stats.py  ...
  notebooks/colab_runner.ipynb      app/app.py
  reports/  artifacts/  (models, FREEZE.lock)   data/ and features/ live on Google Drive, not in git
```

## Phase 0 — Repository, rules and environment

**In plain words.** Set up the empty project so everything later has a home: folders, the rulebook (AGENTS.md), one config file for every setting, pinned library versions, a test runner, and a Colab notebook that clones the repo and mounts Google Drive. No data, no science yet.

**Codex prompt — Phase 0 (copy-paste)**

```
PHASE 0 — Repository setup. Read AGENTS.md first (create it from the text I paste below if missing).
Goal: an empty, testable project skeleton. Do not write any image-processing logic yet.
1. Create the folder layout: src/aidetect/, scripts/, tests/, notebooks/, app/, reports/, artifacts/, docs/.
2. Create config.yaml with: seed: 42; image_size: 256; rapsd_rings: 64; pixel_baseline_size: 32;
   split: {train: 0.70, val: 0.15, test: 0.15}; positive_label: 1 (AI); paths for data_root and
   features_root (default "/content/drive/MyDrive/aidetect"); dataset: {main: "TheKernel01/Tiny-GenImage",
   generator_name: "SD15"}; cv_folds: 5; search_iters: 30; alpha: 0.05.
3. src/aidetect/io_utils.py: load_config(path) -> dict, set_seeds(seed), sha256_of_file(path),
   config_hash(cfg) (sha256 of the sorted YAML dump).
4. requirements.txt with pinned versions for: numpy, scipy, pandas, pyarrow, pillow, scikit-learn,
   xgboost, statsmodels, imagehash, matplotlib, datasets, pyyaml, joblib, streamlit, pytest.
   Pin to the versions you actually installed and tested.
5. tests/test_io_utils.py: config loads, hash is stable across two calls, seeds make numpy reproducible.
6. .github/workflows/tests.yml that runs pytest on push (ubuntu-latest, Python 3.11).
7. notebooks/colab_runner.ipynb: cells to mount Drive, git clone the repo (placeholder URL), pip install -r
   requirements.txt, and run pytest. Add one markdown cell per future phase with a TODO.
8. docs/PHASE_REPORT_TEMPLATE.md with the report template. .gitignore must exclude data/, features/,
   *.parquet, *.npz, artifacts/models/.
Run pytest. Write reports/phase_0_report.md. Commit on branch phase-0. STOP.
```

### Checks you run before proceeding

- pytest passes in Codex and again in Colab (run the notebook top to bottom).
- GitHub Actions shows a green tick on the phase-0 branch.
- config.yaml contains every value listed; no value is hard-coded elsewhere (search the repo for “256”).
- requirements.txt versions match what Colab installed (Colab prints them).

**STOP GATE — end of Phase 0.** Codex writes **reports/phase_0_report.md**, commits, and stops. You run the checks above. Only when every check passes, reply to Codex: **“PROCEED TO PHASE 1”**.

## Phase 1 — Data download (pilot + main)

**In plain words.** Get the images. First a tiny CIFAKE sample (a few hundred images) to prove the pipeline runs end to end. Then the real data: stream Tiny-GenImage from Hugging Face, keep only real photos and Stable Diffusion v1.5 images, and save their original file bytes (not re-saved copies) to Google Drive with a manifest. Saving original bytes matters: re-saving would change JPEG/PNG traces before the audit sees them.

**Codex prompt — Phase 1 (copy-paste)**

```
PHASE 1 — Data acquisition. Follow AGENTS.md. Code only; the user runs the downloads in Colab.
1. src/aidetect/data.py:
   - stream_generator_subset(repo_id, generator_name, split_names, out_dir, max_real, seed):
     use datasets.load_dataset(repo_id, split=..., streaming=True); cast the "image" column to
     datasets.Image(decode=False) so you get the ORIGINAL bytes; resolve generator_name to its integer
     with the dataset's ClassLabel features (do not hard-code 6; read it from .features).
     Keep every row whose generator == generator_name; keep real rows (label 0) with reservoir sampling
     up to max_real. Write each image's original bytes to out_dir/{real|ai}/{source_split}_{index}.{ext},
     where ext comes from the bytes' magic number (JPEG FF D8, PNG 89 50 4E 47), not from the path.
   - write manifest.csv: id, rel_path, label, generator, source_split, ext, n_bytes.
   - print counts per generator for ALL generators seen while streaming (needed to check SD14 vs SD15).
2. scripts/download_main.py: reads config; max_real comes from config (default 4000) so there are more real
   images than AI images to choose from (exact balancing happens in Phase 2). Resumable: skip files on disk.
3. scripts/download_pilot.py: instructions + code to load a CIFAKE sample (500 real + 500 fake) from a
   Kaggle download folder the user provides; same manifest format.
4. Tests (synthetic, no network): magic-number detection, manifest columns, reservoir sampling size,
   resume skips existing files. Mock the dataset iterator with a small list of fake rows.
5. Add Colab cells for both downloads to notebooks/colab_runner.ipynb.
Write reports/phase_1_report.md (include the exact datasets/feature names you relied on and how you
checked them). Commit on branch phase-1. STOP.
```

### Checks you run before proceeding

- Colab printout of counts per generator: write the SD15 and SD14 counts in the team tracker.
- If SD15 has fewer than ~1,000 images: stop and decide with the supervisor (Section 9) before Phase 2.
- Open 5 random files of each class from Drive: they display correctly.
- manifest.csv row count = number of files on Drive.

**STOP GATE — end of Phase 1.** Codex writes **reports/phase_1_report.md**, commits, and stops. You run the checks above. Only when every check passes, reply to Codex: **“PROCEED TO PHASE 2”**.

## Phase 2 — Data audit, de-duplication and frozen splits

**In plain words.** This is the most important phase. Before building any detector, check whether the two classes differ in boring ways — file format, JPEG quality, image size — that a classifier could cheat with. A ‘shortcut probe’ trains a simple model on those boring facts only. If it can tell the classes apart, the frequency results would be meaningless, so you fix the data first. Then remove near-duplicate images and split once into train / validation / test, and lock the split.

**Codex prompt — Phase 2 (copy-paste)**

```
PHASE 2 — Audit, dedup, splits. Follow AGENTS.md.
1. src/aidetect/audit.py:
   - describe_file(path) -> dict: format (from magic bytes), width, height, n_bytes, mode, and for JPEGs
     an estimated quality factor from the quantisation tables (PIL Image.quantization); None for PNG.
   - audit_table(manifest) -> DataFrame with these columns; summary per class: format counts, median
     width/height, JPEG quality distribution.
   - shortcut_probe(df, seed): logistic regression + depth-3 decision tree, 5-fold stratified CV,
     on ONLY [is_jpeg, jpeg_quality (fill -1), width, height, n_bytes, aspect_ratio]. Return mean CV accuracy.
   - normalise_format(img_bytes, target_quality) -> bytes: decode, convert to RGB, re-encode as JPEG at
     target_quality (4:4:4 subsampling). Used ONLY if the probe fails; applied to BOTH classes identically.
2. Decision rule (write it in the report): if probe accuracy > 0.60, the shortcut is present. Then
   re-encode BOTH classes with normalise_format at the median real-image quality, re-run the probe on
   the new files, and report both numbers. Continue only if the probe is <= 0.60.
3. Dedup: imagehash.phash on each image; group images with Hamming distance <= 4 (union-find);
   record group_id. Report how many groups have >1 member.
4. src/aidetect/splits.py: balance classes (randomly drop extra real images, seed 42), then
   StratifiedGroupKFold-style split so a duplicate group never crosses splits, 70/15/15 by label.
   Save data_root/splits.csv (id, label, split, group_id) and artifacts/splits.sha256.
5. scripts/run_audit.py and scripts/make_splits.py; write reports/audit_report.md with tables + plots
   (size histogram per class, JPEG-quality histogram per class).
6. Tests: magic-byte detection, probe returns ~0.5 on random labels, no group crosses splits,
   split proportions within 1 percentage point, balance exact, re-encoding is deterministic.
Write reports/phase_2_report.md. Commit on branch phase-2. STOP.
```

### Checks you run before proceeding

- audit_report.md shows format and JPEG-quality tables for both classes — read them yourself.
- Shortcut probe ≤ 0.60 on the files that will be used (after re-encoding if needed). Write the number down.
- splits.csv: per-split class counts are equal (balanced) and sum to the total.
- No group_id appears in two splits (the test prints 0).
- artifacts/splits.sha256 is committed. From now on, if this hash changes, everything after Phase 2 is invalid.

**STOP GATE — end of Phase 2.** Codex writes **reports/phase_2_report.md**, commits, and stops. You run the checks above. Only when every check passes, reply to Codex: **“PROCEED TO PHASE 3”**.

## Phase 3 — Shared preprocessing function

**In plain words.** Write the one function every image goes through — in training, testing and the app. It turns any image into a 256 × 256 greyscale array with values from 0 to 1. Having exactly one function stops the classic mistake where training and the app prepare images differently.

**Codex prompt — Phase 3 (copy-paste)**

```
PHASE 3 — Preprocessing. Follow AGENTS.md.
1. src/aidetect/preprocess.py: preprocess_image(src, cfg) -> np.ndarray float64 shape (S, S), S = cfg image_size.
   Steps, in this order, identical for every image:
   a) accept a path or raw bytes; decode with Pillow; apply ImageOps.exif_transpose;
   b) convert to RGB then to greyscale "L" (one documented conversion);
   c) centre-crop to a square (side = min(w, h));
   d) if side > S: resize to S x S with Image.Resampling.LANCZOS;
      if side < S: DO NOT upscale — raise ValueError("image smaller than target") (policy from the proposal);
      if side == S: no resize;
   e) convert to float64 and divide by 255.0.
   Raise a clear exception for corrupt or unsupported files.
2. Record the preprocessing settings in a dict PREPROCESS_VERSION = {...} and include it in config_hash.
3. Tests: output shape and range [0,1]; determinism (two calls identical); non-square input is
   centre-cropped (use an image whose left half is black and right half white and check symmetry);
   same pixels saved as PNG vs bytes input give identical output; small image raises; corrupt bytes raise;
   EXIF-rotated image is handled.
4. Colab cell: run preprocess_image on 20 train images per class, show them in a grid, and count how
   many training images raise "smaller than target" (report the count; do not drop them silently).
Write reports/phase_3_report.md. Commit on branch phase-3. STOP.
```

### Checks you run before proceeding

- All tests pass; the Colab grid looks like normal greyscale photos (no stretching).

- Count of “smaller than target” images is written down. If it is more than a few percent, decide: drop them (documented) or set image_size to 224 for everything.
- Search the repo: preprocess_image is the only place that resizes images.

**STOP GATE — end of Phase 3.** Codex writes **reports/phase_3_report.md**, commits, and stops. You run the checks above. Only when every check passes, reply to Codex: **“PROCEED TO PHASE 4”**.

## Phase 4 — Frequency feature module (Deliverable D1)

**In plain words.** Turn the 256 × 256 image into its frequency picture with the 2D FFT and measure four things: how much energy is in fine detail versus smooth areas (energy ratio), how spread out the energy is (entropy), the energy at each distance from the centre in 64 rings (RAPSD), and how many unusually bright dots there are (peaks). The tests use images with a known answer — for example a pure stripe pattern whose bright dot must appear exactly where the maths says.

**Codex prompt — Phase 4 (copy-paste)**

```
PHASE 4 — Frequency features. Follow AGENTS.md.
1. src/aidetect/features.py:
   - power_spectrum(x, window): optional 2D Hann window (np.outer(hanning, hanning)), subtract the image mean,
     F = np.fft.fftshift(np.fft.fft2(x)); return P = |F|**2. Document the centre index (S//2, S//2).
   - radius_map(S): float radius of every bin from the centre.
   - energy_ratio(P, r0): sum(P[r > r0]) / (sum(P[0 < r <= r0]) + eps). DC (r == 0) is excluded.
   - spectral_entropy(P): p = P / P.sum() over non-DC bins; H = -sum(p * log2(p + eps)) / log2(n_bins)
     (normalised to [0, 1]).
   - rapsd(P, K): mean of P in K equal-width rings from r = 1 to r = S//2 (bins beyond S//2, the corners,
     are ignored and this is documented); return log10(mean + eps), length exactly K.
   - peaks(P, k_mad): for each bin with 1 <= r <= S//2, z = (log P - ring median) / (1.4826 * ring MAD + eps);
     a peak is a local 3x3 maximum with z > k_mad; return [count, max z (0 if none)]. Count only one half
     of the spectrum (it is symmetric for real images) so each peak is counted once.
   - extract_frequency_features(x, cfg, r0, k_mad) -> np.ndarray length 1 + 1 + K + 2, plus
     feature_names(cfg) returning matching names. r0 and k_mad are arguments (chosen later on validation data).
2. Tests with known answers:
   - a 256x256 image cos(2*pi*16*col/256) has its spectral maximum at radius 16 and peaks() finds it;
   - pure noise has entropy close to 1 and a flat RAPSD (slope near 0); a smooth gradient has a low
     energy_ratio and a steep RAPSD;
   - output length is exactly 68 for K = 64, all values finite, deterministic, and names match length;
   - an all-constant image does not produce NaN (eps handling).
3. Colab cell: plot the mean log-spectrum and mean RAPSD (with 95% band) of 200 TRAIN images per class.
   Save to reports/figures/. Training data only.
Write reports/phase_4_report.md. Commit on branch phase-4. STOP.
```

### Checks you run before proceeding

- All known-answer tests pass (the stripe test is the key one — it proves the centring and radius are right).
- Look at the mean spectra figure. Bright dots at regular positions in only one class? Note them. Dots at spacings of 1/8 or 1/4 of the image width in the real class only can be JPEG block traces — compare with the Phase 2 audit.
- These figures are exploration, not proof — the report must say so.

**STOP GATE — end of Phase 4.** Codex writes **reports/phase_4_report.md**, commits, and stops. You run the checks above. Only when every check passes, reply to Codex: **“PROCEED TO PHASE 5”**.

## Phase 5 — Pixel baseline and feature extraction for all splits

**In plain words.** Build the comparison baseline (the same image shrunk to 32 × 32 = 1,024 brightness values) and then compute both feature sets for every image. The test-set features are written to a separate sealed folder that nothing reads until Phase 7.

**Codex prompt — Phase 5 (copy-paste)**

```
PHASE 5 — Baseline + extraction. Follow AGENTS.md.
1. src/aidetect/pixel_baseline.py: extract_pixel_features(x, size=32): take the OUTPUT of preprocess_image,
   resize to 32x32 with the same LANCZOS method, flatten to 1,024 values.
2. scripts/extract_features.py --split {train,val,test}: for each image in splits.csv:
   preprocess_image once; store the raw power spectrum summaries needed to recompute features for any r0 /
   k_mad without re-reading images: save per image the RAPSD, the per-ring cumulative energy (so energy_ratio
   for any integer r0 is a lookup), entropy, and the peak z-score map summary for k_mad in [3,4,5,6,8].
   Also save the 1,024 pixel features. Process in batches of 500, write one parquet file per batch to
   features_root/{split}/, and SKIP batches already written (resume after Colab disconnects).
   Each file records config_hash and splits.sha256 in its metadata.
3. Test split: write to features_root/test_sealed/. Code reading that folder must check for
   artifacts/FREEZE.lock and raise PermissionError if it is missing.
4. Tests: baseline length 1,024; lookup-based energy_ratio equals direct energy_ratio for several r0;
   resume skips existing batches; reading test_sealed without the lock raises.
5. Colab cells: run train and val extraction, then test extraction (writing only; never reading).
Write reports/phase_5_report.md with image counts per split and per-batch timings. Commit on phase-5. STOP.
```

### Checks you run before proceeding

- Feature row counts equal the split counts from splits.csv (train, val, test).
- No NaN/Inf in any train/val feature file (the report prints the count: must be 0).
- Try to read test_sealed in Colab: it must raise PermissionError. This proves the seal works.

**STOP GATE — end of Phase 5.** Codex writes **reports/phase_5_report.md**, commits, and stops. You run the checks above. Only when every check passes, reply to Codex: **“PROCEED TO PHASE 6”**.

## Phase 6 — Training and tuning

**In plain words.** Teach the three classifiers on the training data and pick their settings fairly: each model gets the same five cross-validation folds and the same number of setting combinations to try. The validation set is used to pick the energy-ratio radius r0 and the peak threshold. The test set is still untouched.

**Codex prompt — Phase 6 (copy-paste)**

```
PHASE 6 — Training. Follow AGENTS.md. Never touch test_sealed.
1. Choose r0 and k_mad on VALIDATION data only: for r0 in a grid of integers from 8 to 120 (step 8) and k_mad
   in [3,4,5,6,8], build frequency features, fit a default RandomForest on train, score on val; pick the pair
   with the highest val accuracy (ties -> smallest r0, then smallest k_mad). Save to
     artifacts/feature_choice.json.
   Primary model: the frequency-feature model with the best VALIDATION accuracy. Save its name to
   artifacts/primary_model.json now. H1 is tested on this one model (no test-based selection later).
2. src/aidetect/train.py: for each feature set in {frequency, pixel} and model in {svm, rf, xgb}:
   Pipeline(StandardScaler fitted inside CV, model). One StratifiedKFold(5, shuffle=True, random_state=42)
   object shared by all six searches. RandomizedSearchCV with n_iter = cfg search_iters (30) for every model,
   scoring="accuracy", same random_state.
   Search spaces: SVC(kernel="rbf"): C log-uniform 1e-2..1e3, gamma log-uniform 1e-4..1e0;
   RF: n_estimators 200..800, max_depth [None,5,10,20], min_samples_leaf 1..10, max_features ["sqrt",0.3,0.5];
   XGBClassifier: n_estimators 200..800, max_depth 2..8, learning_rate 0.01..0.3, subsample 0.6..1,
   colsample_bytree 0.5..1, eval_metric="logloss". Check every parameter name with help() before use.
3. Refit each best model on train; for SVM wrap in CalibratedClassifierCV(method="sigmoid", cv=5) so it outputs
   calibrated probabilities (Platt scaling). Report validation accuracy, AUC and calibration (Brier score) for all
     six.
4. Save models with joblib to artifacts/models/{featureset}_{model}.joblib plus versions.json
   (python, sklearn, xgboost, numpy versions).
5. Feature importances (RF, XGB, frequency set): top 15 to reports/figures/. Descriptive only.
6. Tests on a small synthetic separable dataset: all six pipelines train and beat 0.9; same seed ->
   same predictions; scaler is fitted only on training folds (assert via a leakage-check helper).
Write reports/phase_6_report.md with a table of CV mean ± sd and val scores. Commit on phase-6. STOP.
```

### Checks you run before proceeding

- Each of the six searches used n_iter = 30 and the same folds (the report prints both).

- Validation accuracy above ~0.98 for everything? Treat it as a warning of a shortcut, not a success: re-check Phase 2 before continuing.
- Team agrees a practical accuracy target for the prototype and writes it in reports/practical_target.md — BEFORE Phase 7 (proposal Section 17).

**STOP GATE — end of Phase 6.** Codex writes **reports/phase_6_report.md**, commits, and stops. You run the checks above. Only when every check passes, reply to Codex: **“PROCEED TO PHASE 7”**.

## Phase 7 — Freeze, single test evaluation, hypothesis tests (Deliverable D2)

**In plain words.** Lock everything, then open the test set exactly once. Compute all metrics for the six models and run the four pre-written statistical tests. The decisions follow automatically from the rules written before the result was known, so nobody can tune the conclusion.

**Codex prompt — Phase 7 (copy-paste)**

```
PHASE 7 — Freeze and test. Follow AGENTS.md.
1. scripts/freeze.py: write artifacts/FREEZE.lock containing sha256 of config.yaml, splits.csv, every file in
   src/aidetect/, feature_choice.json, every model file, practical_target.md, and the UTC time. Refuse to run if
   git has uncommitted changes. Refuse if FREEZE.lock already exists (the test is used once).
2. scripts/evaluate_test.py: verify every hash in FREEZE.lock (abort on mismatch); load test_sealed features;
   for each of the 6 models: accuracy with exact (Clopper-Pearson) 95% CI, precision, recall, F1 (positive = AI),
   ROC-AUC, confusion matrix. Save predictions per image (id, true, pred, prob) for all six.
3. src/aidetect/stats_tests.py (verify each signature with help()):
   - H1: scipy.stats.binomtest(k_correct, n, p=0.5, alternative="greater") for the PRIMARY model in
     primary_model.json; reject H0 if p < 0.05. Report k, n, p, decision (other models: descriptive only).
   - H1a: for each classifier, 2x2 table of (freq correct?, pixel correct?) on the same images;
     statsmodels.stats.contingency_tables.mcnemar(table, exact=True); Holm correction over the 3 p-values
     with statsmodels.stats.multitest.multipletests(method="holm"). One-directional claim only if the frequency
     model has more correct answers and the adjusted p < 0.05.
   - H1b: Cochran's Q (statsmodels ... cochrans_q) on the n x 3 correctness matrix of the three frequency models;
     if p < 0.05, pairwise exact McNemar with Holm over 3 pairs.
   Write every statistic, p-value, adjusted p-value and the decision sentence to reports/results.json + .md.
4. Tests for stats_tests on hand-made data with known answers (e.g. 783 of 1,500 correct gives p ~ 0.047;
   identical predictions give McNemar p = 1).
5. Colab: run freeze.py, then evaluate_test.py ONCE.
Write reports/phase_7_report.md. Commit on branch phase-7. STOP.
```

### Checks you run before proceeding

- FREEZE.lock exists and was committed before evaluate_test.py ran (compare git timestamps).
- results.md contains a decision sentence for H1, H1a and H1b — including if the answer is “not rejected”.
- Re-check one number by hand: accuracy = (TP + TN) / n from the confusion matrix.
- Any change made after this point goes into a section titled “Exploratory analyses”.

**STOP GATE — end of Phase 7.** Codex writes **reports/phase_7_report.md**, commits, and stops. You run the checks above. Only when every check passes, reply to Codex: **“PROCEED TO PHASE 8”**.

## Phase 8 — Streamlit prototype (Deliverable D3)

**In plain words.** A small local web page: upload one image, it runs through the same preprocessing and feature code, the best frozen model gives a label and a score. It says clearly that the score is a model output, not proof.

**Codex prompt — Phase 8 (copy-paste)**

```
PHASE 8 — Prototype. Follow AGENTS.md.
1. app/app.py (Streamlit): st.file_uploader for jpg/jpeg/png/webp; read bytes; call the SAME
   aidetect.preprocess.preprocess_image and the frequency feature function with r0/k_mad from
   feature_choice.json; load the primary model named in artifacts/primary_model.json (chosen on validation data in
   Phase 6, so the test set is never used to pick a model).
2. Show: label (Real / AI-generated), calibrated probability, the log-spectrum image, the RAPSD plot,
   and a fixed note: "Model score for one Stable Diffusion v1.5 dataset; not proof of authenticity; may be
   wrong on other generators, screenshots or compressed images."
3. Reject corrupt files and images smaller than image_size with a clear message (reuse the exceptions).
4. Before predicting, check the models' versions.json against the installed versions and warn on mismatch.
5. Do not store uploads anywhere (process in memory only).
6. Tests: app helper function returns the same probability as the evaluation code for a sample image;
   corrupt file returns an error message, not a crash.
7. README section "Run the demo locally": python -m venv, pip install -r requirements.txt,
   streamlit run app/app.py.
Write reports/phase_8_report.md. Commit on branch phase-8. STOP.
```

### Checks you run before proceeding

- On a laptop: the app starts and classifies 5 test images; the probabilities equal the ones in the saved predictions file.
- A text file renamed to .jpg gives a clear error, not a crash.
- Upload a phone photo or a screenshot: note what happens — this is out-of-distribution and belongs in the limitations.

**STOP GATE — end of Phase 8.** Codex writes **reports/phase_8_report.md**, commits, and stops. You run the checks above. Only when every check passes, reply to Codex: **“PROCEED TO PHASE 9”**.

## Phase 9 — Report pack and reproducibility (supports Deliverable D4)

**In plain words.** Collect every figure and table the final report needs, write a model card, and make sure someone else could rerun the whole thing from the README.

**Codex prompt — Phase 9 (copy-paste)**

```
PHASE 9 — Report pack. Follow AGENTS.md. Do not retrain or re-test anything.
1. scripts/make_figures.py: from saved files only, produce: dataset/audit tables; example spectra per class
   (train); mean RAPSD per class with bands; confusion matrices and ROC curves for all six models;
   results table (accuracy + CI, precision, recall, F1, AUC); hypothesis-test table with decisions;
   feature importances. PNG at 300 dpi to reports/figures/, tables as CSV and Markdown.
2. reports/model_card.md: data (source, licence CC BY-NC-SA 4.0, generator, counts), preprocessing, features,
   chosen r0/k_mad, model, test metrics, intended use, out-of-scope uses, known limitations
   (single generator, no per-category analysis, compression not tested, CIFAKE pilot only).
3. README: full reproduction steps phase by phase, with the Colab notebook order and expected outputs.
4. reports/deviations.md: every place the work differed from the proposal, and why.
Write reports/phase_9_report.md. Commit on branch phase-9. STOP — project build complete.
```

### Checks you run before proceeding

- Every number in the figures matches results.json (spot-check three).
- A team member who did not write the code follows the README on a fresh Colab and gets the same feature files (hash match).
- deviations.md is honest and complete — examiners look for it.

**STOP GATE — end of Phase 9.** The build is complete. Write the final report (D4) from the report pack.

# 7. Difficulties you will face — and fixes

Ranked by damage. The first four can give you a **high accuracy that is not real**, which is worse than a low accuracy because it leads to a wrong conclusion. Fix those first.

| # | Difficulty | What you will see | Fix |
|---|---|---|---|
| 1 | JPEG-vs-PNG shortcut. Published work (“Fake or JPEG?”, ECCV 2024 Workshops) shows GenImage real images are JPEG while generated ones are PNG, and detectors learn that difference. | Very high accuracy (often 95%+) that collapses when AI images are JPEG-compressed. | Phase 2 shortcut probe; if it fails, re-encode both classes to JPEG at the real images’ median quality and re-probe. Report the probe before and after. |
| 2 | Image-size shortcut. The same paper shows detectors learn the generator’s output size. | Accuracy depends on original image size; probe uses width/height. | Use SD v1.5 (similar size to real photos); centre-crop + one resize; include size in the probe; never upscale. |
| 3 | Leakage: near-duplicate images across splits, scaler fitted on all data, or peeking at the test set. | Test accuracy much higher than CV; suspiciously perfect scores. | phash grouping in Phase 2; scaler inside the CV pipeline; sealed test folder + FREEZE.lock + single-use evaluate script. |
| 4 | Validation results that look too good (≥ 98%). | Celebrating too early. | Treat as an alarm: inspect misclassified-vs-correct metadata; rerun the probe; check spectra for 8-pixel JPEG grid peaks. |
| 5 | Small subset → low statistical power. SD v1.5 may give only ~1,500–2,200 AI images (Likely, from partial statistics). | A test set of ~500–650 images; a modest real effect may not reach significance. | Use the threshold table (Section 8); report confidence intervals; if too small, ask the supervisor about the full GenImage SD v1.4 subset (scope change, Section 18 of the proposal). |
| 6 | CIFAKE is 32 × 32: a 32 × 32 spectrum has only 16 radial steps and the downsizing removes high-frequency traces. | Pilot features look weak or meaningless. | Use CIFAKE only to test the code path. Never report pilot numbers as results. |
| 7 | Colab free limits: sessions end, idle disconnects, RAM ~12–13 GB (Likely), GPU not guaranteed. | Lost progress mid-extraction; “session crashed” on big arrays. | CPU is enough (no GPU needed). Batches of 500 with resume; everything saved to Drive; stream the dataset instead of loading 8.36 GB; store features, not decoded images. Kaggle notebooks as backup. |
| 8 | Codex free-plan usage limits are tight and vary. | Codex stops mid-phase or refuses new tasks until the limit resets. | One phase per task; prompts are self-contained; AGENTS.md carries the rules so nothing must be re-explained; commit often so a reset loses nothing. While waiting, run the Colab checks. Use ChatGPT Free chat to explain errors. |
| 9 | AI-written code that looks right but is wrong (invented parameters, off-by-one FFT centre). | Silent wrong numbers rather than errors. | Known-answer tests (stripe image → peak at radius 16; 783/1,500 → p ≈ 0.047); AGENTS.md rule 7 (check signatures); paste exact error text back to Codex; never accept code without a test run. |
| 10 | Codex cannot see your dataset (it lives on Drive). | Codex asks for data or writes untestable code. | Tests use synthetic images; real data runs only in Colab cells Codex writes for you. |
| 11 | FFT details: DC term dominating, log(0), centre index for even sizes, edge effects. | Entropy ≈ 0, NaN features, cross-shaped artefacts in spectra. | Subtract mean and exclude DC; eps everywhere; fftshift with centre (S//2, S//2) tested; optional Hann window fixed before testing. |
| 12 | Unfair classifier comparison (one model tuned more). | H1b result reflects tuning effort, not the model. | Same folds object, same n_iter = 30, same seed for all six searches (Phase 6 check). |

| # | Difficulty | What you will see | Fix |
|---|---|---|---|
| 13 | SVM gives no usable confidence; slow on big pixel features. | Raw decision scores shown as “confidence”. | CalibratedClassifierCV(sigmoid); n is only a few thousand so RBF-SVM is fine on CPU. |
| 14 | Laptop vs Colab version mismatch breaks loading models. | Unpickling errors or different predictions in the app. | Pinned requirements.txt; versions.json saved with models; app warns on mismatch; use the same Python minor version. |
| 15 | Prototype users upload phone photos, screenshots, WhatsApp images. | Confident but wrong answers. | Fixed disclaimer in the app; list as out-of-distribution in limitations; do not promise general detection. |
| 16 | Null result (accuracy near 50%). | Feeling the project failed. | It is a planned, valid outcome (proposal Section 5). The decision rules and report template already handle it. |
| 17 | No per-category labels in the Hugging Face copy. | Cannot report per-category accuracy. | State as a limitation; content is roughly matched because GenImage used the same ImageNet classes. |
| 18 | Licence: Tiny-GenImage is CC BY-NC-SA 4.0. | Unclear whether the demo or a published copy is allowed. | Academic, non-commercial use is fine; attribute the dataset and GenImage; do not redistribute images in the repo. |
| 19 | Three people, one repo: merge conflicts and drift. | Overwritten work, two versions of a function. | Branch per phase, one owner per phase, PR review by one other member, merge only after the STOP-gate checks. |
| 20 | Literature review time; risk of citing papers no one has read. | Weak or invented references. | Start with the “Fake or JPEG?” paper and the GenImage paper (both checked to exist); cite only what a member has read. |

# 8. Statistics quick reference

Minimum number correct on a balanced test set for a one-sided exact binomial test against 50% to give $p < 0.05$. Computed with scipy.stats; the 200 / 500 / 1,000 / 2,000 rows match the proposal's table exactly, and the 1,500 row agrees with the normal approximation ($750 + 1.645 \times \sqrt{1500} / 2 \approx 781.9$, so 782–783).

| Test images | Minimum correct | Minimum accuracy | Exact p at that count |
|---|---|---|---|
| 200 | 113 | 56.5% | 0.038 |
| 500 | 269 | 53.8% | 0.049 |
| 600 (expected for SD v1.5 subset) | 321 | 53.5% | 0.047 |
| 1,000 | 527 | 52.7% | 0.047 |
| 1,500 (proposal assumption) | 783 | 52.2% | 0.047 |
| 2,000 | 1,038 | 51.9% | 0.047 |

| Hypothesis | Function (checked in statsmodels / scipy) | Decision |
|---|---|---|
| H1 vs H0 | scipy.stats.binomtest(k, n, p=0.5, alternative='greater') | Reject H0 if $p < 0.05$ |
| H1a | statsmodels.stats.contingency_tables.mcnemar(table, exact=True) per classifier; statsmodels.stats.multitest.multipletests(pvals, method='holm') | Frequency better if it has more correct and Holm-adjusted $p < 0.05$ |
| H1b | statsmodels.stats.contingency_tables.cochrans_q(x) on the n × 3 correctness matrix; then pairwise McNemar + Holm | Models differ if Q-test $p < 0.05$; pairwise results say which |

**Significant ≠ useful.** With 600 test images, 54% accuracy would reject H0 but would be a poor detector. Agree the practical target before Phase 7 and report both answers.

# 9. Decisions to confirm with the supervisor

| Decision | Default in this PRD | Needed before |
|---|---|---|
| Main dataset and generator | Tiny-GenImage, SD v1.5 subset; CIFAKE as pilot | Phase 1 |
| What to do if SD v1.5 has < ~1,000 images | Ask about full GenImage SD v1.4 (scope change) | Phase 2 |
| Format fix if the shortcut probe fails | Re-encode both classes to JPEG at the real median quality | Phase 2 end |
| Images smaller than 256 px | Exclude and document (no upscaling) | Phase 3 end |
| Shortcut-probe pass threshold | Assumption: CV accuracy ≤ 0.60 on metadata only. If the supervisor sets a stricter bar (e.g. 0.55), Phase 2 may need the format fix more often | Phase 2 |
| Primary model for H1 | Frequency model with best validation accuracy, recorded before the test | Phase 6 end |
| Pixel baseline | 32 × 32 greyscale, 1,024 values | Phase 5 |
| Practical accuracy target for the prototype | To be agreed and written in practical_target.md | Phase 7 |
| Prototype framework | Streamlit, local | Phase 8 |

# 10. Sources checked for this PRD

- Team documents: *Proposal Draft & Scope Definition* v0.1 and *Assignment 4 — Methodology & Analysis* (MCA2554).

- OpenAI Help Centre, “Using Codex with your ChatGPT plan” — Codex included across plans including Free; limits vary by plan. help.openai.com/en/articles/11369540
- Google Colab FAQ — free resources not guaranteed; limits fluctuate. research.google.com/colaboratory/faq.html
- Hugging Face dataset card and statistics: TheKernel01/Tiny-GenImage (splits, generator labels, licence, image-size statistics).
- Zhu et al., *GenImage: A Million-Scale Benchmark for Detecting AI-Generated Image*, NeurIPS 2023 Datasets & Benchmarks. arxiv.org/abs/2306.08571
- Grommelt, Weiss, Pfreundt, Keuper, *Fake or JPEG? Revealing Common Biases in Generated Image Detection Datasets*, ECCV 2024 Workshops. arxiv.org/abs/2403.17608
- Free-tier session/RAM figures for Colab and Kaggle: third-party 2026 guides (marked Likely where used). Re-check limits on the day you start.